



Persistence des données sous android avec SQLite



Plan

I.	QU'EST-CE QUE SQLITE	1
II.	SQLITEOPENHELPER	1
II.1.	QUELQUES METHODES	1
II.2.	EXEMPLE	1
III.	CONTENTVALUES	1
III.1.	QUELQUES METHODES	1
III.2.	EXEMPLE	1
IV.	SQLITEDATABASE	2
IV.1.	QUELQUES METHODES	2
IV.2.	EXEMPLES	2
IV.2.1.	<i>insert</i>	2
IV.2.2.	<i>delete</i>	2
IV.2.3.	<i>update</i>	2
IV.2.4.	<i>query</i>	3
IV.2.5.	<i>rawQuery</i>	3
V.	CURSOR	3
V.1.	QUELQUES METHODES	3
V.2.	EXEMPLE	3
VI.	UTILISATION DU ? DANS LES REQUETES	4
VI.1.	UTILITE	4
VI.2.	EXEMPLES	4
VI.2.1.	<i>? dans delete</i>	4
VI.2.2.	<i>? dans update</i>	4
VI.2.3.	<i>? dans query</i>	4
VI.2.4.	<i>? dans rawQuery</i>	4
VII.	ETAPES A SUIVRE	4

I. Qu'est-ce que SQLite

SQLite est une bibliothèque open source écrite en C qui propose un moteur de base de données relationnelle accessible par le langage SQL. Android inclut une implémentation de SQLite.

II. SQLiteOpenHelper

SQLiteOpenHelper est une classe abstraite du package `android.database.sqlite` qui permet de gérer la création de la base et ses versions, généralement on utilise une sous classe qui implémente les deux méthodes abstraites `onCreate` et `onUpgrade`.

II.1. Quelques méthodes

- *SQLiteOpenHelper(Context context, String name, CursorFactory factory, int version)*
- *constructor(context: Context?, name: String?, factory: CursorFactory?, version: Int)*
- *fun getWritableDatabase: SQLiteDatabase*
- *fun getReadableDatabase: SQLiteDatabase*

II.2. Exemple

```
package com.bib
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteDatabase.CursorFactory
import android.database.sqlite.SQLiteOpenHelper
class SQLiteBib(
    context: Context?,
    name: String?,
    factory: CursorFactory?,
    version: Int
) : SQLiteOpenHelper(context, name, factory, version) {

    override fun onCreate(db: SQLiteDatabase) {
        val sql = "create table Livre (id INTEGER PRIMARY KEY AUTOINCREMENT, titre text
NOT NULL, nbpage INTEGER );"
        db.execSQL(sql)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        // Logique de mise à niveau de la base de données si nécessaire
    }
}
```

III. ContentValues

ContentValues est une classe du package `android.content` qui permet de stocker un ensemble de paires (clé, valeur).

III.1. Quelques méthodes

- *constructor()*
- *fun put(key: String, value: Int?)*
- *fun put(key: String, value: Float?)*
- *fun put(key: String, value: String?)*

III.2. Exemple

```
val v = ContentValues().apply {
    put("titre", "Android2")
    put("nbpage", 170)
}
```

IV. SQLiteDatabase

SQLiteDatabase est une classe du package android.database.sqlite qui permet de gérer une base de données SQLite.

IV.1. Quelques méthodes

- *fun close()*
- *fun insert(table: String?, nullColumnHack: String?, values: ContentValues?): Long*
- *fun update(table: String?, values: ContentValues?, whereClause: String?, whereArgs: Array<String?>?): Int*
- *fun delete(table: String?, whereClause: String?, whereArgs: Array<String?>?): Int*
- *fun query(table: String?, columns: Array<String?>?, selection: String?, selectionArgs: Array<String?>?, groupBy: String?, having: String?, orderBy: String?, limit: String?): Cursor?*
- *fun rawQuery(sql: String?, selectionArgs: Array<String?>?): Cursor?*
- *fun execSQL(sql: String?)*

IV.2. Exemples

IV.2.1. insert

```
val titre = "Android2"
val nbPage = 170
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
val v = ContentValues().apply {
    put("titre", titre)
    put("nbpage", nbPage)
}
db.insert("Livre", null, v)
db.close()
```

IV.2.2. delete

```
val id = 1
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
db.delete("Livre", "id=$id", null)
db.close()
```

IV.2.3. update

```
val id = 1
val titre = "Android II"
val nbPage = 180
val v = ContentValues().apply {
    put("titre", titre)
    put("nbpage", nbPage)
}
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
db.update("Livre", v, "id=$id", null)
db.close()
```

IV.2.4. query

```
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
val c = db.query(
    "Livre",
    arrayOf("id", "titre", "nbpage"),
    null, null, null, null, null
)
```

IV.2.5. rawQuery

```
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
val sql = "Select * From Livre"
val c = db.rawQuery(sql, null)
```

V. Cursor

Cursor est une interface du package `android.database.sqlite` qui permet l'accès à un résultat d'une requête sur une base SQLite.

V.1. Quelques méthodes

- *abstract fun close()*
- *abstract fun getFloat(columnIndex: Int): Float*
- *abstract fun getInt(columnIndex: Int): Int*
- *abstract fun getString(columnIndex: Int): String*
- *abstract fun getBlob(columnIndex: Int): ByteArray*
- *abstract fun moveToNext(): Boolean*
- *abstract fun moveToPosition(position: Int): Boolean*
- *abstract fun moveToPrevious(): Boolean*

V.2. Exemple

```
// Supposant que la classe Livre existe : class Livre(val id: Int, val titre:
String, val nbpage: Int)
while (c.moveToNext()) {
    val id = c.getInt(0)
    val titre = c.getString(1)
    val nbpage = c.getInt(2)
    val l = Livre(id, titre, nbpage)
}
c.close()
```

VI. Utilisation du ? dans les requêtes

VI.1. Utilité

Dans la clause Where des requêtes update, delete, query et rawQuery on peut utiliser des ? dans les emplacements des valeurs de tests des conditions. Par la suite les valeurs sont passées dans un tableau de String. L'utilisation du ? permet d'obtenir une écriture simple des requêtes pour minimiser les erreurs et simplifier leurs correction.

VI.2. Exemples

VI.2.1. ? dans delete

```
val id = 1
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
db.delete("Livre", "id=?", arrayOf(id.toString()))
db.close()
```

VI.2.2. ? dans update

```
val id = 1
val titre = "Android II"
val nbPage = 180
val v = ContentValues().apply {
    put("titre", titre)
    put("nbpage", nbPage)
}
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
db.update("Livre", v, "id = ?", arrayOf(id.toString()))
db.close()
```

VI.2.3. ? dans query

Exécuter la requête : « SELECT * From Livre Where titre like %android% And nbpage>50 ».

```
val titre = "android"
val minNbPage = 50
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
val c = db.query(
    "Livre",
    arrayOf("id", "titre", "nbpage"),
    "titre like ? And nbpage>?",
    arrayOf("%$titre%", minNbPage.toString()),
    null, null, null
)
```

VI.2.4. ? dans rawQuery

Exécuter la requête : "Select * From Livre Where titre like %android% And nbpage>50".

```
val titre = "android"
val minNbPage = 50
val b = SQLiteBib(this, "bib.db", null, 1)
val db = b.writableDatabase
val sql = "Select * From Livre Where titre like ? And nbpage>?"
val c = db.rawQuery(sql, arrayOf("%$titre%", minNbPage.toString()))
```

VII. Etapes à suivre

Lors de la programmation d'une application android qui utilise une base de données SQLite, suivre les étapes suivantes :

- Déclarer une classe équivalente à chaque table de la base SQLite. Le nombre d'attributs de chaque classe est égal à celui des champs de la table.
- Déclarer une sous classe de la classe SQLiteOpenHelper et redéfinir le constructeur, la méthode onCreate et la méthode onUpgrade.
- Utiliser les méthodes de la classe SQLiteDatabase pour exécuter les requêtes.
- Utiliser la classe ContentValues pour remplir les paires clé/valeur des requêtes INSERT et UPDATE.
- Utiliser la classe Cursor pour parcourir les résultats des requêtes de type SELECT.